

CS 157A Team 5 Project DB Model and Design: RAGdoll

Team Members:

Ethan Vu (Team Lead): ethan.vu@sjsu.edu

Geo Lalu: geo.lalu@sjsu.edu

Naman Kumar: naman.kumar@sjsu.edu

Table of Contents

Functional Requirements	3
Entities, Attributes, Relationships with Multiplicity	6
ERD	11
Relational Schemas	12
MySQL Workbench Tables	14

Functional Requirements

Users:

- General Users
 - Can create an account using a username, email, and password.
 - Can log in with verified credentials while passwords are stored separately as salted password hashes.
 - Can subscribe to a tier that determines which models they are allowed to access.
 - Can upload source documents for retrieval-augmented generation.
 - Can create conversations, submit prompts, view generated responses, and review previous conversation history.
- Paid Users
 - Paid users are general users who have completed a payment for an upgraded tier.
 - Paid users may access additional local models or expanded usage privileges based on their tier.
 - Their payment and tier information is stored in the database for access control.
- Admins
 - Admins are a subclass of Users and can perform user, model, tier, and document management actions.
 - Can review or manage uploaded documents when needed.
 - Can use audit logs to track system actions such as account creation, document uploads, prompts, payments, and admin changes.

Functions:

Create User

- Users create an account by entering username, email, and password.
- The system stores user profile fields in Users and authentication fields in User_Hashes.
- The system records the creation event in Audit_Log.

User Authentication

- Users log in using email or username and a password.
- The application compares a computed hash against User_Hashes.password_hash and never stores the plaintext password.
- Login failure messages are returned by the application, not by exposing database details.

Tier and Payment Tracking

- Users can be assigned one active tier through User_Tiers.
- Payments store the user, selected tier, payment status, and payment date.
- A tier grants access to one or more models through Tier_Model_Access.

Upload Document

- Users upload documents with file name and file type metadata.
- Documents are tied to the uploading user.
- Admins may manage uploaded documents and those actions are recorded.

Split Document into Chunks

- Each document is split into multiple chunks.
- Each chunk stores the chunk text and its embedding vector.

Create Conversation

- Users create conversations to group related prompts and responses.

- Each conversation has a title and creation date.
- Queries and responses are linked back to the conversation for history.

Submit Prompt or Query

- A user enters a query_text prompt.
- The system creates an embedding_vector for the query.
- The query is associated with a user and conversation.

Retrieve Relevant Chunks

- The RAG application compares the query embedding to stored chunk embeddings.
- Retrieved chunks are stored in Query_Retrieved_Chunks.
- The model receives the top retrieved chunks as context.

Generate Response

- A selected model is prompted by a query.
- The generated response text is stored with the query_id and model_id.
- Conversation_Responses links responses to conversation history.

Audit System Actions

- The system stores important actions such as uploads, payments, prompts, and admin reviews.
- Audit_Log can retain the action even if a user is removed by setting user_id to NULL in that table

Entities, Attributes, Relationships with Multiplicity

Entities:

Users

Users contain the basic account data per user and is identified by the primary key user_id, and it contains the username, email, and date attributes. When an account is created these basic identifying details are stored in this table. The user password isn't stored, but its hash is, along with a random unique salt. Each user has a different tier they can access through either a payment or elevated access. They have the ability to make queries and view their conversations. Every action a user makes is stored in the audit log.

User_Hashes

User_Hashes are the weak entity set of Users, including all the security attributes of password_hash and salt. Since each User_Hash must relate to one and one only User, once the User no longer exists, the User_Hash is no longer necessary. Thus the relationship between the two entity sets are weak, with User_Hashes dependent on the User.

Admins

Admins are a subclass of Users that can manage documents. They inherit the same primary user_id key and additionally have a company_id that represents the company the admin is assigned to. Admin accounts are not created through the same login flows as users (they are manually inputted). However they share all other same data including hashes. The primary purpose of an Admin is to add/delete/modify the model specific documents to be served via RAG to the normal users.

Tiers

Tiers is the entity set containing all the different types of tiers for users. Each tier has a tier_id, tier_name and price. A payment is necessary to set a tier. This table exists so the different plans and levels of access to models can be changed over time. If the User is an Admin, the tier functionality is optional as Admins have access to the database anyway.

Payments

Payments is the entity set that keeps track of individual payments processed by users to upgrade tiers. Its primary key is payment_id and its foreign keys are user_id and tier_id from Users and Tiers respectively. It also has two additional attributes: status and date. In the final application literal payments will not be implemented, but simulated ones will.

Models

Models is the entity set for the various LLMs used. Its primary key is model_id, and its attributes are model_name, model_type, and model_path. They are prompted by Users and deemed accessible by Tiers, and they generate responses. In the application there is a separate local LLM server that handles all requests.

Documents

Documents are the internal model documents that are uploaded and controlled by Admins. Their primary key is document_id, have the foreign key user_id, and the attributes file_name and file_type. These documents are split into chunks to be used in RAG. The document uploading is done via the admin panel.

Chunks

Chunks are the split portions of documents. These splits are generated after upload to work with the RAG system. The primary key is chunk_id, the foreign key is document_id, and its attributes

are `chunk_text` and `embedding_vector`. The embedding vector is used by RAG for information retrieval done via user queries.

Conversations

Conversations is the entity set of all conversations of all users. Its primary key is `conversation_id` and its foreign key is `user_id`. Its attributes are title and date. From the application interface, the user can see a list of all past conversations and have them returned as a list.

Queries

Queries is the entity set that stores each individual prompt submitted by a user. It is a strong entity set, not a weak entity set, because each query has its own primary key, `query_id`. Each query is created by one user and belongs to one conversation, but it does not depend on the conversation for its identity. Its attributes are `query_id`, `user_id`, `query_text`, `embedding_vector`, and date. The `embedding_vector` is used by the RAG system to compare the user's query against stored chunk embeddings and retrieve relevant context.

Responses

Responses is the entity set that stores the generated answer returned by a model. It is also a strong entity set, not a weak entity set, because each response has its own primary key, `response_id`. Each response answers one query and is generated by one model. Its attributes are `response_id`, `query_id`, `model_id`, `response_text`, and date. The `query_id` connects the response back to the user's original prompt, while `model_id` records which model generated the response.

Relationships:

Triggers

Triggers is a one to many relationship between Users and Audit_Log. It uses the User's user_id and Audit_Log's log_id.

Pays

Pays is a one to many relationship between Users and Payments. It uses the Users' user_id and Payments' payment_id.

Sets

Sets is a one to many relationship between Tiers and Payments. It uses the Tiers' tier_id and Payments' payment_id.

Has

Has is a many to many relationship between Users and Tiers. It uses the Users' user_id and Tiers' tier_id.

Creates

Creates is a one to many relationship between Users and Queries. It uses the Users' user_id and Queries' query_id.

Owns

Owns is a one to many relationship between Users and Conversations. It uses the Users' user_id and Conversations' conversation_id.

Accesses

Accesses is a many to many relationship between Tiers and Models. It uses the Tiers' tier_id and Models' model_id.

Prompts

Prompts is a many to many relationship between Queries and Models. It uses the Queries' query_id and Models' model_id.

Generates

Generates is a one to many relationship between Models and Responses. It uses the Models' model_id and Responses' response_id.

Retrieves

Retrieves is a many to many relationship between Queries and Chunks. It uses the Queries' query_id and Chunks' chunk_id.

Answers

Answers is a one to many relationship between Queries and Responses. It uses the Queries' query_id and Responses' response_id.

Contains_Response

Contains_Response is a many to many relationship between Conversations and Responses. It uses the Conversations' conversation_id and Responses' response_id.

Contains_Query

Contains_Query is a one to many relationship between Conversations and Queries. It uses the Conversations' conversation_id and Queries' query_id.

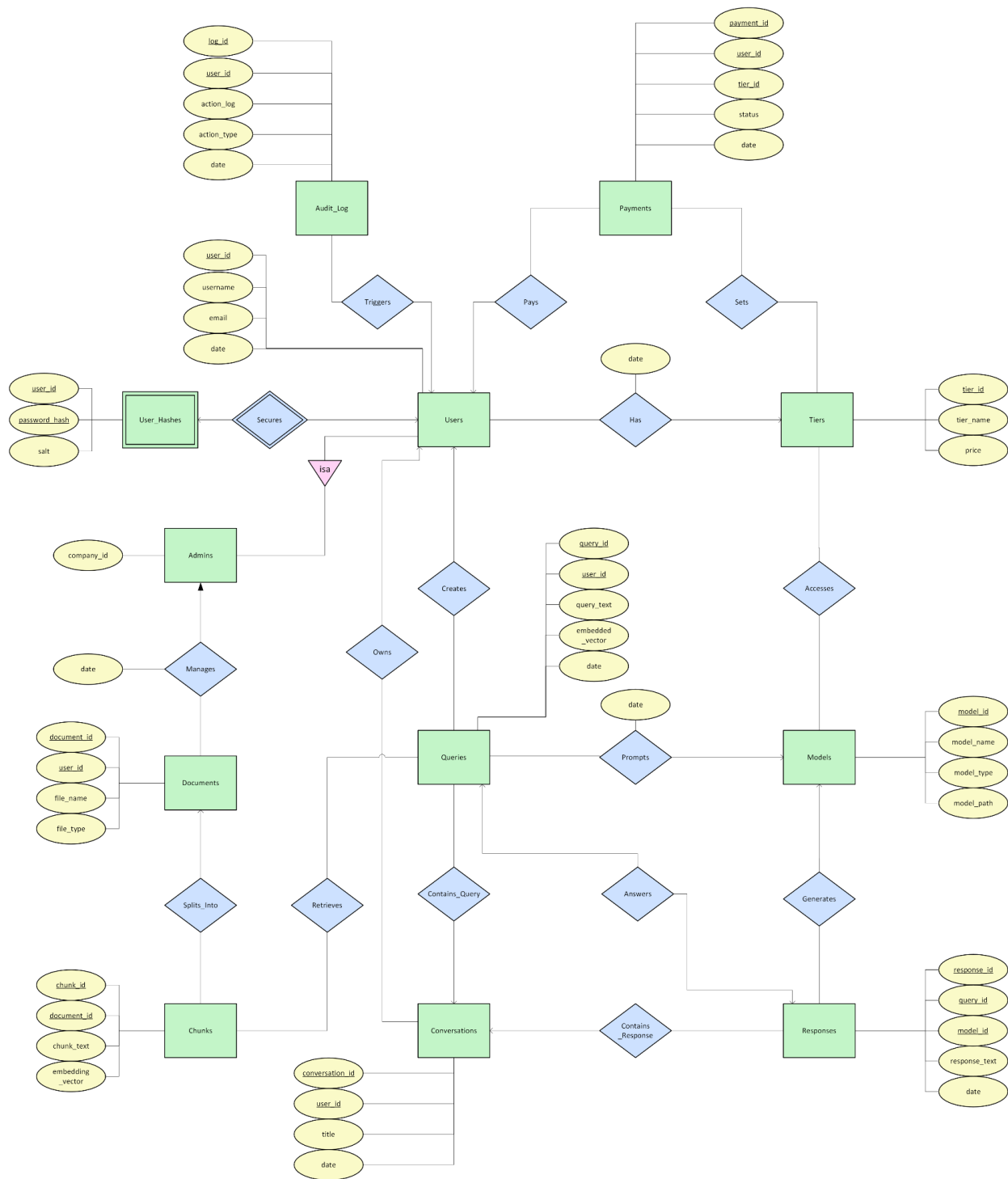
Manages

Manages is a many to many relationship between Admins and Documents. It uses the Admins' user_id and Documents' document_id.

Splits_Into

Splits_Into is a one to many relationship between Documents and Chunks. It uses the Documents' document_id and Chunks' chunk_id.

ERD



Relational Schemas

Entities

Users(user_id PK, username, email, created_at)

User_Hashes(user_id PK, password_hash CK, salt)

Admins(user_id PK/FK, company_id, admin_email)

Tiers(tier_id PK, tier_name, price)

Payments(payment_id PK, user_id FK, tier_id FK, status, payment_date)

Models(model_id PK, model_name, model_type, model_path)

Documents(document_id PK, user_id FK, file_name, file_type, uploaded_at)

Chunks(chunk_id PK, document_id FK, chunk_text, embedding_vector)

Conversations(conversation_id PK, user_id FK, title, created_at)

Queries(query_id PK, user_id FK, query_text, embedding_vector, created_at)

Responses(response_id PK, query_id FK, model_id FK, response_text, created_at)

Audit_Log(log_id PK, user_id FK, action_log, action_type, action_date)

Relationships

Triggers(user_id FK, log_id FK)

Pays(user_id FK, payment_id FK)

Sets(tier_id FK, payment_id FK)

Has(user_id FK, tier_id FK, assigned_at)

Creates(user_id FK, query_id FK)

Owns(user_id FK, conversation_id FK)

Access(tier_id FK, model_id FK)

Prompts(query_id FK, model_id FK, prompted_at)

Generates(model_id FK, response_id FK)

Retrieves(query_id FK, document_id FK, chunk_id FK)

Answers(query_id FK, response_id FK)

Contains_Response(conversation_id FK, response_id FK)

Contains_Query(conversation_id FK, query_id FK)

Manages(admin_user_id FK, document_id FK, managed_at)

Splits_Into(document_id FK, chunk_id FK)

MySQL Workbench Tables

SCHEMAS

Filter objects

- bars-drinkers-beers
- ragdoll
- ragdoll_db**
 - Tables
 - access
 - admins
 - answers
 - audit_log
 - chunks
 - contains_query
 - contains_response
 - conversations
 - creates
 - documents
 - generates
 - has
 - manages
 - models
 - owns
 - payments
 - pays
 - prompts
 - queries
 - responses
 - retrieves
 - sets
 - splits_into
 - tiers
 - triggers
 - user_hashes
 - users
 - Views
 - Stored Procedures
 - Functions
- sys
- vu

SQL Editor

```

1 • USE ragdoll_db;
2 • SELECT * FROM `Users`;
3 • SELECT * FROM `User_Hashes`;
4 • SELECT * FROM `Admins`;
5 • SELECT * FROM `Tiers`;
6 • SELECT * FROM `Payments`;
7 • SELECT * FROM `Models`;
8 • SELECT * FROM `Documents`;
9 • SELECT * FROM `Chunks`;
10 • SELECT * FROM `Conversations`;
11 • SELECT * FROM `Queries`;
12 • SELECT * FROM `Responses`;
13 • SELECT * FROM `Audit_Log`;
14
15 • SELECT * FROM `Triggers`;
16 • SELECT * FROM `Pays`;
17 • SELECT * FROM `Sets`;
18 • SELECT * FROM `Has`;
19 • SELECT * FROM `Creates`;
20 • SELECT * FROM `Owns`;

```

Result Grid

	user_id	username	email	created_at
▶	1	ethanv	ethan.vu@sjsu.edu	2026-07-01 10:00:00
	2	geolalu	geo.lalu@sjsu.edu	2026-07-01 10:05:00
	3	namank	naman.kumar@sjsu.edu	2026-07-01 10:10:00
	4	sophial	sophia.lee@example.com	2026-07-01 10:15:00
	5	michaelc	michael.chen@example.com	2026-07-01 10:20:00
	6	priyap	priya.patel@example.com	2026-07-01 10:25:00
	7	jordanw	jordan.wong@example.com	2026-07-01 10:30:00
	8	mariag	maria.garcia@example.com	2026-07-01 10:35:00
	9	omarh	omar.hassan@example.com	2026-07-01 10:40:00
	10	lisaw	lisa.wilson@example.com	2026-07-01 10:45:00
	11	admin_a...	avery.admin@ragdoll.local	2026-07-01 09:00:00
	12	admin_bl...	blake.admin@ragdoll.local	2026-07-01 09:05:00
	13	admin_c...	casey.admin@ragdoll.local	2026-07-01 09:10:00
	14	admin_d...	devon.admin@ragdoll.local	2026-07-01 09:15:00
	15	admin_e...	emery.admin@ragdoll.local	2026-07-01 09:20:00
	16	admin_finn	finn.admin@ragdoll.local	2026-07-01 09:25:00
	17	admin_gale	gale.admin@ragdoll.local	2026-07-01 09:30:00
	18	admin_h...	harper.admin@ragdoll.local	2026-07-01 09:35:00
	19	admin_iris	iris.admin@ragdoll.local	2026-07-01 09:40:00
	20	admin_riley	riley.admin@ragdoll.local	2026-07-01 09:45:00
*	NULL	NULL	NULL	NULL

SCHEMAS

Filter objects

- bars-drinkers-beers
- ragdoll
- ragdoll_db**
 - Tables
 - access
 - admins
 - answers
 - audit_log
 - chunks
 - contains_query
 - contains_response
 - conversations
 - creates
 - documents
 - generates
 - has
 - manages
 - models
 - owns
 - payments
 - pays
 - prompts
 - queries
 - responses
 - retrieves
 - sets
 - splits_into
 - tiers
 - triggers
 - user_hashes
 - users
 - Views
 - Stored Procedures
 - Functions
- sys
- vu

Limit to 5000 rows

```

1 • USE ragdoll_db;
2 • SELECT * FROM `Users`;
3 • SELECT * FROM `User_Hashes`;
4 • SELECT * FROM `Admins`;
5 • SELECT * FROM `Tiers`;
6 • SELECT * FROM `Payments`;
7 • SELECT * FROM `Models`;
8 • SELECT * FROM `Documents`;
9 • SELECT * FROM `Chunks`;
10 • SELECT * FROM `Conversations`;
11 • SELECT * FROM `Queries`;
12 • SELECT * FROM `Responses`;
13 • SELECT * FROM `Audit_Log`;
14
15 • SELECT * FROM `Triggers`;
16 • SELECT * FROM `Pays`;
17 • SELECT * FROM `Sets`;
18 • SELECT * FROM `Has`;
19 • SELECT * FROM `Creates`;
20 • SELECT * FROM `Owns`;

```

Result Grid | Filter Rows: | Edit: | Export/1

	user_id	password_hash	salt
1	97b410a6f4e2647967390dfd80a59f1da77c3ff5...	salt_ethanv_1	
2	031c67f9059d4fe09b465e401e5b718e88a9141...	salt_geolalu_2	
3	54b69da06e8f889b5454873d78e6c78ae682e9...	salt_namank_3	
4	bbada7a901b68a72fcb99eb282d218d50aab51c...	salt_sophial_4	
5	c53aaf18207e6e9b97546a8c2a48b253fac78c9...	salt_michaelc_5	
6	52b6fbe8c57d4b4355d37ba20e4aa09ea46a59...	salt_priyap_6	
7	408d33c34f490f45a4ee2e35326ff78e13f8a55c...	salt_jordanw_7	
8	652aeccc68f72e10a5d66a64e87fb1c05033dfc8...	salt_mariag_8	
9	c99cb9d5d5fbd05dfac3224ce9009de0df20d18...	salt_omarh_9	
10	a95299ec717464bdc27d8cbe0893bbc05f91601...	salt_lisaw_10	
11	9e04ce1d00e139b2e2e993a04505406b888d69...	salt_admin_av...	
12	c4c7ca7d72aa904e87facabc4731c3008f799d7b...	salt_admin_bla...	
13	76689315a0bc0c7fcb8274343f06e47d53521db...	salt_admin_ca...	
14	4345c12056811568cd223a80e0ed39e1d4338a...	salt_admin_de...	
15	ada89f68cdfc2c6ebff5247a146b0aa98e139552...	salt_admin_em...	
16	2b305237bbc160a336ed1299cf3271379dd494d...	salt_admin_fin...	
17	64d9b9fed778883c77ba9c8ea13f91a5bedf12e...	salt_admin_gal...	
18	a5534220017215cfbf83a52563bace12d6cf108...	salt_admin_ha...	
19	0d5219bea0fa87519ed64d8f3b3783d0abf7977...	salt_admin_iris...	
20	8f0adf098a933aa5cb2b74c8bb22bf20e3ac832...	salt_admin_rile...	
*	NULL	NULL	NULL

SCHEMAS

Filter objects

- bars-drinkers-beers
- ragdoll
 - ragdoll_db**
 - Tables
 - access
 - admins
 - answers
 - audit_log
 - chunks
 - contains_query
 - contains_response
 - conversations
 - creates
 - documents
 - generates
 - has
 - manages
 - models
 - owns
 - payments
 - pays
 - prompts
 - queries
 - responses
 - retrieves
 - sets
 - splits_into
 - tiers
 - triggers
 - user_hashes
 - users
 - Views
 - Stored Procedures
 - Functions
 - sys
 - vu

Limit to 5000 rows

```

1 • USE ragdoll_db;
2 • SELECT * FROM `Users`;
3 • SELECT * FROM `User_Hashes`;
4 • SELECT * FROM `Admins`;
5 • SELECT * FROM `Tiers`;
6 • SELECT * FROM `Payments`;
7 • SELECT * FROM `Models`;
8 • SELECT * FROM `Documents`;
9 • SELECT * FROM `Chunks`;
10 • SELECT * FROM `Conversations`;
11 • SELECT * FROM `Queries`;
12 • SELECT * FROM `Responses`;
13 • SELECT * FROM `Audit_Log`;
14
15 • SELECT * FROM `Triggers`;
16 • SELECT * FROM `Pays`;
17 • SELECT * FROM `Sets`;
18 • SELECT * FROM `Has`;
19 • SELECT * FROM `Creates`;
20 • SELECT * FROM `Owns`;
  
```

Result Grid | Filter Rows: | Edit:

	user_id	company_id	admin_email
▶	11	RAGDOLL001	avery.admin@ragdoll.local
	12	RAGDOLL002	blake.admin@ragdoll.local
	13	RAGDOLL003	casey.admin@ragdoll.local
	14	RAGDOLL004	devon.admin@ragdoll.local
	15	RAGDOLL005	emery.admin@ragdoll.local
	16	RAGDOLL006	finn.admin@ragdoll.local
	17	RAGDOLL007	gale.admin@ragdoll.local
	18	RAGDOLL008	harper.admin@ragdoll.local
	19	RAGDOLL009	iris.admin@ragdoll.local
	20	RAGDOLL010	riley.admin@ragdoll.local
*	NULL	NULL	NULL

SCHEMAS

Filter objects

- bars-drinkers-beers
- ragdoll
- ragdoll_db**
 - Tables
 - access
 - admins
 - answers
 - audit_log
 - chunks
 - contains_query
 - contains_response
 - conversations
 - creates
 - documents
 - generates
 - has
 - manages
 - models
 - owns
 - payments
 - pays
 - prompts
 - queries
 - responses
 - retrieves
 - sets
 - splits_into
 - tiers
 - triggers
 - user_hashes
 - users
 - Views
 - Stored Procedures
 - Functions
- sys
- vu

1 • **USE** ragdoll_db;

2 • **SELECT** * **FROM** `Users`;

3 • **SELECT** * **FROM** `User\_Hash`;

4 • **SELECT** * **FROM** `Admins`;

5 • **SELECT** * **FROM** `Tiers`;

6 • **SELECT** * **FROM** `Payments`;

7 • **SELECT** * **FROM** `Models`;

8 • **SELECT** * **FROM** `Documents`;

9 • **SELECT** * **FROM** `Chunks`;

10 • **SELECT** * **FROM** `Conversations`;

11 • **SELECT** * **FROM** `Queries`;

12 • **SELECT** * **FROM** `Responses`;

13 • **SELECT** * **FROM** `Audit\_Log`;

14

15 • **SELECT** * **FROM** `Triggers`;

16 • **SELECT** * **FROM** `Pays`;

17 • **SELECT** * **FROM** `Sets`;

18 • **SELECT** * **FROM** `Has`;

19 • **SELECT** * **FROM** `Creates`;

20 • **SELECT** * **FROM** `Owns`;

Result Grid | Filter Rows:

	tier_id	tier_name	price
▶	1	Free	0.00
	2	Student	4.99
	3	Standard	9.99
	4	Plus	14.99
	5	Premium	19.99
	6	Research	24.99
	7	Developer	29.99
	8	Team	49.99
	9	Enterprise	99.99
	10	Admin Access	0.00
*	NULL	NULL	NULL

Navigator

SCHEMAS

Filter objects

- bars-drinkers-beers
- ragdoll
- ragdoll_db**
 - Tables
 - access
 - admins
 - answers
 - audit_log
 - chunks
 - contains_query
 - contains_response
 - conversations
 - creates
 - documents
 - generates
 - has
 - manages
 - models
 - owns
 - payments
 - pays
 - prompts
 - queries
 - responses
 - retrieves
 - sets
 - splits_into
 - tiers
 - triggers
 - user_hashes
 - users
 - Views
 - Stored Procedures
 - Functions
- sys
- vu

SQL File 1 x database

Limit to 5000 rows

```

1 • USE ragdoll_db;
2 • SELECT * FROM `Users`;
3 • SELECT * FROM `User_Hashes`;
4 • SELECT * FROM `Admins`;
5 • SELECT * FROM `Tiers`;
6 • SELECT * FROM `Payments`;
7 • SELECT * FROM `Models`;
8 • SELECT * FROM `Documents`;
9 • SELECT * FROM `Chunks`;
10 • SELECT * FROM `Conversations`;
11 • SELECT * FROM `Queries`;
12 • SELECT * FROM `Responses`;
13 • SELECT * FROM `Audit_Log`;
14
15 • SELECT * FROM `Triggers`;
16 • SELECT * FROM `Pays`;
17 • SELECT * FROM `Sets`;
18 • SELECT * FROM `Has`;
19 • SELECT * FROM `Creates`;
20 • SELECT * FROM `Owns`;

```

Result Grid | Filter Rows: | Edit: | E

	payment_id	user_id	tier_id	status	payment_date
▶	1	1	1	Free Tier	2026-07-02 12:00:00
	2	2	2	Completed	2026-07-02 12:05:00
	3	3	3	Completed	2026-07-02 12:10:00
	4	4	4	Completed	2026-07-02 12:15:00
	5	5	5	Completed	2026-07-02 12:20:00
	6	6	6	Completed	2026-07-02 12:25:00
	7	7	7	Completed	2026-07-02 12:30:00
	8	8	8	Completed	2026-07-02 12:35:00
	9	9	9	Completed	2026-07-02 12:40:00
	10	10	5	Completed	2026-07-02 12:45:00
*	NULL	NULL	NULL	NULL	NULL

Navigator: SCHEMAS

Filter objects

- bars-drinkers-beers
- ragdoll
- ragdoll_db**
 - Tables
 - access
 - admins
 - answers
 - audit_log
 - chunks
 - contains_query
 - contains_response
 - conversations
 - creates
 - documents
 - generates
 - has
 - manages
 - models
 - owns
 - payments
 - pays
 - prompts
 - queries
 - responses
 - retrieves
 - sets
 - splits_into
 - tiers
 - triggers
 - user_hashes
 - users
 - Views
 - Stored Procedures
 - Functions
- sys
- vu

SQL File 11* x database

Limit to 5000 rows

```

1 • USE ragdoll_db;
2 • SELECT * FROM `Users`;
3 • SELECT * FROM `User_Hashes`;
4 • SELECT * FROM `Admins`;
5 • SELECT * FROM `Tiers`;
6 • SELECT * FROM `Payments`;
7 • SELECT * FROM `Models`;
8 • SELECT * FROM `Documents`;
9 • SELECT * FROM `Chunks`;
10 • SELECT * FROM `Conversations`;
11 • SELECT * FROM `Queries`;
12 • SELECT * FROM `Responses`;
13 • SELECT * FROM `Audit_Log`;
14
15 • SELECT * FROM `Triggers`;
16 • SELECT * FROM `Pays`;
17 • SELECT * FROM `Sets`;
18 • SELECT * FROM `Has`;
19 • SELECT * FROM `Creates`;
20 • SELECT * FROM `Owns`;

```

Result Grid

Filter Rows: Edit: Export/Import

	model_id	model_name	model_type	model_path
▶	1	Llama 3.1 8B	LLM	/models/llama-3.1-8b.gguf
	2	Gemma 2B	LLM	/models/gemma-2b.gguf
	3	Mistral 7B	LLM	/models/mistral-7b.gguf
	4	Phi 3 Mini	LLM	/models/phi-3-mini.gguf
	5	Mythos Vision	Multimodal	/models/mythos-vision.safetensors
	6	MiniLM Embedder	Embedding	/models/all-MiniLM-L6-v2
	7	BGE Small Embedder	Embedding	/models/bge-small-en-v1.5
	8	CodeLlama 7B	Code	/models/codellama-7b.gguf
	9	TinyLlama 1.1B	LLM	/models/tinyllama-1.1b.gguf
	10	Whisper Base	Speech	/models/whisper-base.bin
*	NULL	NULL	NULL	NULL

SCHEMAS

Filter objects

- bars-drinkers-beers
- ragdoll
 - ragdoll_db
 - Tables
 - access
 - admins
 - answers
 - audit_log
 - chunks
 - contains_query
 - contains_response
 - conversations
 - creates
 - documents
 - generates
 - has
 - manages
 - models
 - owns
 - payments
 - pays
 - prompts
 - queries
 - responses
 - retrieves
 - sets
 - splits_into
 - tiers
 - triggers
 - user_hashes
 - users
 - Views
 - Stored Procedures
 - Functions
 - sys
 - vu

Limit to 5000 rows

```

1 • USE ragdoll_db;
2 • SELECT * FROM `Users`;
3 • SELECT * FROM `User_Hashes`;
4 • SELECT * FROM `Admins`;
5 • SELECT * FROM `Tiers`;
6 • SELECT * FROM `Payments`;
7 • SELECT * FROM `Models`;
8 • SELECT * FROM `Documents`;
9 • SELECT * FROM `Chunks`;
10 • SELECT * FROM `Conversations`;
11 • SELECT * FROM `Queries`;
12 • SELECT * FROM `Responses`;
13 • SELECT * FROM `Audit_Log`;
14
15 • SELECT * FROM `Triggers`;
16 • SELECT * FROM `Pays`;
17 • SELECT * FROM `Sets`;
18 • SELECT * FROM `Has`;
19 • SELECT * FROM `Creates`;
20 • SELECT * FROM `Owns`;
  
```

Result Grid | Filter Rows: | Edit: | Export/Import:

	document_id	user_id	file_name	file_type	uploaded_at
▶	1	11	rag_overview.pdf	PDF	2026-07-03 09:00:00
	2	12	mysql_notes.txt	TXT	2026-07-03 09:10:00
	3	13	project_requirements.pdf	PDF	2026-07-03 09:20:00
	4	14	vector_search_guide.pdf	PDF	2026-07-03 09:30:00
	5	15	prompt_engineering.md	MD	2026-07-03 09:40:00
	6	1	ethan_research_notes.pdf	PDF	2026-07-03 10:00:00
	7	2	geo_database_summary.docx	DOCX	2026-07-03 10:10:00
	8	3	naman_model_access.txt	TXT	2026-07-03 10:20:00
	9	4	sophia_api_reference.pdf	PDF	2026-07-03 10:30:00
	10	5	michael_testing_plan.md	MD	2026-07-03 10:40:00
*	NULL	NULL	NULL	NULL	NULL

SCHEMAS

Filter objects

- bars-drinkers-beers
- ragdoll
 - ragdoll_db**
 - Tables
 - access
 - admins
 - answers
 - audit_log
 - chunks
 - contains_query
 - contains_response
 - conversations
 - creates
 - documents
 - generates
 - has
 - manages
 - models
 - owns
 - payments
 - pays
 - prompts
 - queries
 - responses
 - retrieves
 - sets
 - splits_into
 - tiers
 - triggers
 - user_hashes
 - users
 - Views
 - Stored Procedures
 - Functions
 - sys
 - vu

Limit to 5000 rows

```

1 • USE ragdoll_db;
2 • SELECT * FROM `Users`;
3 • SELECT * FROM `User_Hashes`;
4 • SELECT * FROM `Admins`;
5 • SELECT * FROM `Tiers`;
6 • SELECT * FROM `Payments`;
7 • SELECT * FROM `Models`;
8 • SELECT * FROM `Documents`;
9 • SELECT * FROM `Chunks`;
10 • SELECT * FROM `Conversations`;
11 • SELECT * FROM `Queries`;
12 • SELECT * FROM `Responses`;
13 • SELECT * FROM `Audit_Log`;
14
15 • SELECT * FROM `Triggers`;
16 • SELECT * FROM `Pays`;
17 • SELECT * FROM `Sets`;
18 • SELECT * FROM `Has`;
19 • SELECT * FROM `Creates`;
20 • SELECT * FROM `Owns`;
  
```

Result Grid | Filter Rows: | Edit: | Export/Import: |

	chunk_id	document_id	chunk_text	embedding_vector
▶	1	1	Retrieval augmented generation uses retrieved ...	[0.12, 0.25, 0.33, 0.41]
	2	2	MySQL stores relational data using tables, prim...	[0.05, 0.15, 0.28, 0.36]
	3	3	The project requires updated functional require...	[0.21, 0.30, 0.40, 0.52]
	4	4	Vector search compares query embeddings with...	[0.18, 0.22, 0.39, 0.44]
	5	5	Prompt engineering improves model output by p...	[0.14, 0.27, 0.31, 0.49]
	6	6	The RAGdoll application stores user prompts an...	[0.19, 0.24, 0.38, 0.50]
	7	7	Database normalization reduces duplicated data...	[0.08, 0.17, 0.33, 0.42]
	8	8	Premium tiers can unlock additional local models ...	[0.22, 0.31, 0.45, 0.58]
	9	9	API documentation explains how the application...	[0.11, 0.29, 0.37, 0.47]
	10	10	Testing plans include authentication, document ...	[0.16, 0.26, 0.34, 0.46]
*	NULL	NULL	NULL	NULL

SCHEMAS

Filter objects

- bars-drinkers-beers
- ragdoll
 - ragdoll_db
 - Tables
 - access
 - admins
 - answers
 - audit_log
 - chunks
 - contains_query
 - contains_response
 - conversations
 - creates
 - documents
 - generates
 - has
 - manages
 - models
 - owns
 - payments
 - pays
 - prompts
 - queries
 - responses
 - retrieves
 - sets
 - splits_into
 - tiers
 - triggers
 - user_hashes
 - users
 - Views
 - Stored Procedures
 - Functions
 - sys
 - vu

Limit to 5000 rows

```

1 • USE ragdoll_db;
2 • SELECT * FROM `Users`;
3 • SELECT * FROM `User_Hashes`;
4 • SELECT * FROM `Admins`;
5 • SELECT * FROM `Tiers`;
6 • SELECT * FROM `Payments`;
7 • SELECT * FROM `Models`;
8 • SELECT * FROM `Documents`;
9 • SELECT * FROM `Chunks`;
10 • SELECT * FROM `Conversations`;
11 • SELECT * FROM `Queries`;
12 • SELECT * FROM `Responses`;
13 • SELECT * FROM `Audit_Log`;
14
15 • SELECT * FROM `Triggers`;
16 • SELECT * FROM `Pays`;
17 • SELECT * FROM `Sets`;
18 • SELECT * FROM `Has`;
19 • SELECT * FROM `Creates`;
20 • SELECT * FROM `Owns`;
  
```

Result Grid | Filter Rows: | Edit: | Export/Imp

	conversation_id	user_id	title	created_at
▶ 1	1	1	RAG System Help	2026-07-04 14:00:00
2	2	2	Database Design Questions	2026-07-04 14:30:00
3	3	3	Model Access Discussion	2026-07-04 15:00:00
4	4	4	Vector Search Setup	2026-07-04 15:30:00
5	5	5	Prompt Writing Help	2026-07-04 16:00:00
6	6	6	Conversation History Review	2026-07-04 16:30:00
7	7	7	Normalization Practice	2026-07-04 17:00:00
8	8	8	Payment Tier Question	2026-07-04 17:30:00
9	9	9	Local Model Server	2026-07-04 18:00:00
10	10	10	Testing Checklist	2026-07-04 18:30:00
*	NULL	NULL	NULL	NULL

SCHEMAS

Filter objects

- bars-drinkers-beers
- ragdoll
 - ragdoll_db
 - Tables
 - access
 - admins
 - answers
 - audit_log
 - chunks
 - contains_query
 - contains_response
 - conversations
 - creates
 - documents
 - generates
 - has
 - manages
 - models
 - owns
 - payments
 - pays
 - prompts
 - queries
 - responses
 - retrieves
 - sets
 - splits_into
 - tiers
 - triggers
 - user_hashes
 - users
 - Views
 - Stored Procedures
 - Functions
 - sys
 - vu

Limit to 5000 rows

```

1 • USE ragdoll_db;
2 • SELECT * FROM `Users`;
3 • SELECT * FROM `User_Hashes`;
4 • SELECT * FROM `Admins`;
5 • SELECT * FROM `Tiers`;
6 • SELECT * FROM `Payments`;
7 • SELECT * FROM `Models`;
8 • SELECT * FROM `Documents`;
9 • SELECT * FROM `Chunks`;
10 • SELECT * FROM `Conversations`;
11 • SELECT * FROM `Queries`;
12 • SELECT * FROM `Responses`;
13 • SELECT * FROM `Audit_Log`;
14
15 • SELECT * FROM `Triggers`;
16 • SELECT * FROM `Pays`;
17 • SELECT * FROM `Sets`;
18 • SELECT * FROM `Has`;
19 • SELECT * FROM `Creates`;
20 • SELECT * FROM `Owns`;

```

Result Grid

query_id	user_id	query_text	embedding_vector	created_at
1	1	How does RAG retrieve relevant chunks?	[0.13, 0.24, 0.35, 0.48]	2026-07-04 14:01:00
2	2	How should I store chunks in MySQL?	[0.07, 0.16, 0.29, 0.37]	2026-07-04 14:31:00
3	3	Which models can premium users access?	[0.22, 0.31, 0.45, 0.58]	2026-07-04 15:01:00
4	4	What is the purpose of an embedding vector?	[0.18, 0.23, 0.36, 0.43]	2026-07-04 15:31:00
5	5	How can prompts be written more clearly?	[0.15, 0.28, 0.32, 0.50]	2026-07-04 16:01:00
6	6	Where is conversation history stored?	[0.20, 0.25, 0.39, 0.51]	2026-07-04 16:31:00
7	7	Why should database tables be normalized?	[0.09, 0.18, 0.34, 0.43]	2026-07-04 17:01:00
8	8	How are paid tiers connected to model access?	[0.23, 0.32, 0.46, 0.59]	2026-07-04 17:31:00
9	9	How does the local model server receive reques...	[0.12, 0.30, 0.38, 0.48]	2026-07-04 18:01:00
10	10	What should be tested before demo day?	[0.17, 0.27, 0.35, 0.47]	2026-07-04 18:31:00
NULL	NULL	NULL	NULL	NULL

SCHEMAS

Filter objects

- bars-drinkers-beers
- ragdoll
 - ragdoll_db**
 - Tables
 - access
 - admins
 - answers
 - audit_log
 - chunks
 - contains_query
 - contains_response
 - conversations
 - creates
 - documents
 - generates
 - has
 - manages
 - models
 - owns
 - payments
 - pays
 - prompts
 - queries
 - responses
 - retrieves
 - sets
 - splits_into
 - tiers
 - triggers
 - user_hashes
 - users
 - Views
 - Stored Procedures
 - Functions
 - sys
 - vu

Limit to 5000 rows

```

1 • USE ragdoll_db;
2 • SELECT * FROM `Users`;
3 • SELECT * FROM `User_Hashes`;
4 • SELECT * FROM `Admins`;
5 • SELECT * FROM `Tiers`;
6 • SELECT * FROM `Payments`;
7 • SELECT * FROM `Models`;
8 • SELECT * FROM `Documents`;
9 • SELECT * FROM `Chunks`;
10 • SELECT * FROM `Conversations`;
11 • SELECT * FROM `Queries`;
12 • SELECT * FROM `Responses`;
13 • SELECT * FROM `Audit_Log`;
14
15 • SELECT * FROM `Triggers`;
16 • SELECT * FROM `Pays`;
17 • SELECT * FROM `Sets`;
18 • SELECT * FROM `Has`;
19 • SELECT * FROM `Creates`;
20 • SELECT * FROM `Owns`;

```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Cor

	response_id	query_id	model_id	response_text	created_at
▶	1	1	1	The system compares the query embedding aga...	2026-07-04 14:01:10
	2	2	1	Store each chunk with a chunk_id, document_id...	2026-07-04 14:31:10
	3	3	2	Premium users can access models connected to ...	2026-07-04 15:01:10
	4	4	6	An embedding vector is a numeric representatio...	2026-07-04 15:31:10
	5	5	3	A clear prompt states the task, gives useful con...	2026-07-04 16:01:10
	6	6	4	Conversation history is stored through Convers...	2026-07-04 16:31:10
	7	7	1	Normalization helps avoid duplicated facts and k...	2026-07-04 17:01:10
	8	8	2	Paid tier access is modeled by connecting Users ...	2026-07-04 17:31:10
	9	9	8	The application sends the prompt and retrieved ...	2026-07-04 18:01:10
	10	10	9	Before demo day, test account creation, login, ...	2026-07-04 18:31:10
*	NULL	NULL	NULL	NULL	NULL

Navigator

SCHEMAS

Filter objects

- bars-drinkers-beers
- ragdoll
 - ragdoll_db
 - Tables
 - access
 - admins
 - answers
 - audit_log
 - chunks
 - contains_query
 - contains_response
 - conversations
 - creates
 - documents
 - generates
 - has
 - manages
 - models
 - owns
 - payments
 - pays
 - prompts
 - queries
 - responses
 - retrieves
 - sets
 - splits_into
 - tiers
 - triggers
 - user_hashes
 - users
 - Views
 - Stored Procedures
 - Functions
 - sys
 - vu

SQL File 11* x database

Limit to 5000 rows

```

1 • USE ragdoll_db;
2 • SELECT * FROM `Users`;
3 • SELECT * FROM `User_Hashes`;
4 • SELECT * FROM `Admins`;
5 • SELECT * FROM `Tiers`;
6 • SELECT * FROM `Payments`;
7 • SELECT * FROM `Models`;
8 • SELECT * FROM `Documents`;
9 • SELECT * FROM `Chunks`;
10 • SELECT * FROM `Conversations`;
11 • SELECT * FROM `Queries`;
12 • SELECT * FROM `Responses`;
13 • SELECT * FROM `Audit_Log`;
14
15 • SELECT * FROM `Triggers`;
16 • SELECT * FROM `Pays`;
17 • SELECT * FROM `Sets`;
18 • SELECT * FROM `Has`;
19 • SELECT * FROM `Creates`;
20 • SELECT * FROM `Owns`;

```

Result Grid

Filter Rows:

Edit: Export/Import: Wrap Cell Cor

log_id	user_id	action_log	action_type	action_date
1	1	User account created for ethanv	CREATE_USER	2026-07-01 10:00:00
2	2	Payment completed for Student tier	PAYMENT	2026-07-02 12:05:00
3	11	Admin uploaded rag_overview.pdf	UPLOAD_DOCUMENT	2026-07-03 09:00:00
4	1	User submitted query about RAG retrieval	SUBMIT_QUERY	2026-07-04 14:01:00
5	3	Payment completed for Standard tier	PAYMENT	2026-07-02 12:10:00
6	12	Admin uploaded mysql_notes.txt	UPLOAD_DOCUMENT	2026-07-03 09:10:00
7	5	User created prompt writing conversation	CREATE_CONVERSATION	2026-07-04 16:00:00
8	8	User asked about paid model access	SUBMIT_QUERY	2026-07-04 17:31:00
9	14	Admin reviewed vector_search_guide.pdf	ADMIN_REVIEW	2026-07-03 09:35:00
10	10	User submitted testing checklist query	SUBMIT_QUERY	2026-07-04 18:31:00
NULL	NULL	NULL	NULL	NULL

Navigator: database

SQL File 11* x database

SCHEMAS

Filter objects

- bars-drinkers-beers
- ragdoll
- ragdoll_db**
 - Tables
 - access
 - admins
 - answers
 - audit_log
 - chunks
 - contains_query
 - contains_response
 - conversations
 - creates
 - documents
 - generates
 - has
 - manages
 - models
 - owns
 - payments
 - pays
 - prompts
 - queries
 - responses
 - retrieves
 - sets
 - splits_into
 - tiers
 - triggers
 - user_hashes
 - users
 - Views
 - Stored Procedures
 - Functions
- sys
- vu

```

1 • USE ragdoll_db;
2 • SELECT * FROM `Users`;
3 • SELECT * FROM `User_Hashes`;
4 • SELECT * FROM `Admins`;
5 • SELECT * FROM `Tiers`;
6 • SELECT * FROM `Payments`;
7 • SELECT * FROM `Models`;
8 • SELECT * FROM `Documents`;
9 • SELECT * FROM `Chunks`;
10 • SELECT * FROM `Conversations`;
11 • SELECT * FROM `Queries`;
12 • SELECT * FROM `Responses`;
13 • SELECT * FROM `Audit_Log`;
14
15 • SELECT * FROM `Triggers`;
16 • SELECT * FROM `Pays`;
17 • SELECT * FROM `Sets`;
18 • SELECT * FROM `Has`;
19 • SELECT * FROM `Creates`;
20 • SELECT * FROM `Owns`;
21 • SELECT * FROM ``;

```

Result Grid | Filter Rows:

	user_id	log_id
▶	1	1
	1	4
	2	2
	3	5
	5	7
	8	8
	10	10
	11	3
	12	6
	14	9
*	NULL	NULL

Navigator

SQL File 11* x database

SCHEMAS

Filter objects

- bars-drinkers-beers
- ragdoll
- ragdoll_db**
 - Tables
 - access
 - admins
 - answers
 - audit_log
 - chunks
 - contains_query
 - contains_response
 - conversations
 - creates
 - documents
 - generates
 - has
 - manages
 - models
 - owns
 - payments
 - pays
 - prompts
 - queries
 - responses
 - retrieves
 - sets
 - splits_into
 - tiers
 - triggers
 - user_hashes
 - users
 - Views
 - Stored Procedures
 - Functions
- sys
- vu

```

1 • USE ragdoll_db;
2 • SELECT * FROM `Users`;
3 • SELECT * FROM `User_Hashes`;
4 • SELECT * FROM `Admins`;
5 • SELECT * FROM `Tiers`;
6 • SELECT * FROM `Payments`;
7 • SELECT * FROM `Models`;
8 • SELECT * FROM `Documents`;
9 • SELECT * FROM `Chunks`;
10 • SELECT * FROM `Conversations`;
11 • SELECT * FROM `Queries`;
12 • SELECT * FROM `Responses`;
13 • SELECT * FROM `Audit_Log`;
14
15 • SELECT * FROM `Triggers`;
16 • SELECT * FROM `Pays`;
17 • SELECT * FROM `Sets`;
18 • SELECT * FROM `Has`;
19 • SELECT * FROM `Creates`;
20 • SELECT * FROM `Owns`;

```

Result Grid

Filter Rows:

	user_id	payment_id
▶	1	1
	2	2
	3	3
	4	4
	5	5
	6	6
	7	7
	8	8
	9	9
	10	10
*	NULL	NULL

Navigator

SCHEMAS

Filter objects

- bars-drinkers-beers
- ragdoll
- ragdoll_db**
 - Tables
 - access
 - admins
 - answers
 - audit_log
 - chunks
 - contains_query
 - contains_response
 - conversations
 - creates
 - documents
 - generates
 - has
 - manages
 - models
 - owns
 - payments
 - pays
 - prompts
 - queries
 - responses
 - retrieves
 - sets
 - splits_into
 - tiers
 - triggers
 - user_hashes
 - users
 - Views
 - Stored Procedures
 - Functions
- sys
- vu

SQL File 1 x Database

```

1 • USE ragdoll_db;
2 • SELECT * FROM `Users`;
3 • SELECT * FROM `User_Hashes`;
4 • SELECT * FROM `Admins`;
5 • SELECT * FROM `Tiers`;
6 • SELECT * FROM `Payments`;
7 • SELECT * FROM `Models`;
8 • SELECT * FROM `Documents`;
9 • SELECT * FROM `Chunks`;
10 • SELECT * FROM `Conversations`;
11 • SELECT * FROM `Queries`;
12 • SELECT * FROM `Responses`;
13 • SELECT * FROM `Audit_Log`;
14
15 • SELECT * FROM `Triggers`;
16 • SELECT * FROM `Pays`;
17 • SELECT * FROM `Sets`;
18 • SELECT * FROM `Has`;
19 • SELECT * FROM `Creates`;
20 • SELECT * FROM `Owns`;
21 • SELECT * FROM `Splits`;

```

Result Grid

	tier_id	payment_id
▶	1	1
	2	2
	3	3
	4	4
	5	5
	5	10
	6	6
	7	7
	8	8
	9	9
*	NULL	NULL

Navigator: SQL File 11* x database

SCHEMAS

Filter objects

- bars-drinkers-beers
- ragdoll
- ragdoll_db**
 - Tables
 - access
 - admins
 - answers
 - audit_log
 - chunks
 - contains_query
 - contains_response
 - conversations
 - creates
 - documents
 - generates
 - has
 - manages
 - models
 - owns
 - payments
 - pays
 - prompts
 - queries
 - responses
 - retrieves
 - sets
 - splits_into
 - tiers
 - triggers
 - user_hashes
 - users
 - Views
 - Stored Procedures
 - Functions
- sys
- vu

```

1 • USE ragdoll_db;
2 • SELECT * FROM `Users`;
3 • SELECT * FROM `User_Hashes`;
4 • SELECT * FROM `Admins`;
5 • SELECT * FROM `Tiers`;
6 • SELECT * FROM `Payments`;
7 • SELECT * FROM `Models`;
8 • SELECT * FROM `Documents`;
9 • SELECT * FROM `Chunks`;
10 • SELECT * FROM `Conversations`;
11 • SELECT * FROM `Queries`;
12 • SELECT * FROM `Responses`;
13 • SELECT * FROM `Audit_Log`;
14
15 • SELECT * FROM `Triggers`;
16 • SELECT * FROM `Pays`;
17 • SELECT * FROM `Sets`;
18 • SELECT * FROM `Has`;
19 • SELECT * FROM `Creates`;
20 • SELECT * FROM `Owns`;

```

Result Grid | Filter Rows:

	user_id	tier_id	assigned_at
▶	1	1	2026-07-02 12:00:00
	2	2	2026-07-02 12:05:00
	3	3	2026-07-02 12:10:00
	4	4	2026-07-02 12:15:00
	5	5	2026-07-02 12:20:00
	6	6	2026-07-02 12:25:00
	7	7	2026-07-02 12:30:00
	8	8	2026-07-02 12:35:00
	9	9	2026-07-02 12:40:00
	10	5	2026-07-02 12:45:00
*	NULL	NULL	NULL

Navigator: SQL File 11* x database

SCHEMAS

Filter objects

- bars-drinkers-beers
- ragdoll
 - ragdoll_db
 - Tables
 - access
 - admins
 - answers
 - audit_log
 - chunks
 - contains_query
 - contains_response
 - conversations
 - creates
 - documents
 - generates
 - has
 - manages
 - models
 - owns
 - payments
 - pays
 - prompts
 - queries
 - responses
 - retrieves
 - sets
 - splits_into
 - tiers
 - triggers
 - user_hashes
 - users
 - Views
 - Stored Procedures
 - Functions
 - sys
 - vu

```

1 • USE ragdoll_db;
2 • SELECT * FROM `Users`;
3 • SELECT * FROM `User_Hashes`;
4 • SELECT * FROM `Admins`;
5 • SELECT * FROM `Tiers`;
6 • SELECT * FROM `Payments`;
7 • SELECT * FROM `Models`;
8 • SELECT * FROM `Documents`;
9 • SELECT * FROM `Chunks`;
10 • SELECT * FROM `Conversations`;
11 • SELECT * FROM `Queries`;
12 • SELECT * FROM `Responses`;
13 • SELECT * FROM `Audit_Log`;
14
15 • SELECT * FROM `Triggers`;
16 • SELECT * FROM `Pays`;
17 • SELECT * FROM `Sets`;
18 • SELECT * FROM `Has`;
19 • SELECT * FROM `Creates`;
20 • SELECT * FROM `Owns`;
21 • SELECT * FROM ``;
  
```

Result Grid | Filter Rows:

	user_id	query_id
▶	1	1
	2	2
	3	3
	4	4
	5	5
	6	6
	7	7
	8	8
	9	9
	10	10
*	NULL	NULL

Navigator: database

SQL File 11* x database

SCHEMAS

Filter objects

- bars-drinkers-beers
- ragdoll
- ragdoll_db**
 - Tables
 - access
 - admins
 - answers
 - audit_log
 - chunks
 - contains_query
 - contains_response
 - conversations
 - creates
 - documents
 - generates
 - has
 - manages
 - models
 - owns
 - payments
 - pays
 - prompts
 - queries
 - responses
 - retrieves
 - sets
 - splits_into
 - tiers
 - triggers
 - user_hashes
 - users
 - Views
 - Stored Procedures
 - Functions
- sys
- vu

```

1 • USE ragdoll_db;
2 • SELECT * FROM `Users`;
3 • SELECT * FROM `User_Hashes`;
4 • SELECT * FROM `Admins`;
5 • SELECT * FROM `Tiers`;
6 • SELECT * FROM `Payments`;
7 • SELECT * FROM `Models`;
8 • SELECT * FROM `Documents`;
9 • SELECT * FROM `Chunks`;
10 • SELECT * FROM `Conversations`;
11 • SELECT * FROM `Queries`;
12 • SELECT * FROM `Responses`;
13 • SELECT * FROM `Audit_Log`;
14
15 • SELECT * FROM `Triggers`;
16 • SELECT * FROM `Pays`;
17 • SELECT * FROM `Sets`;
18 • SELECT * FROM `Has`;
19 • SELECT * FROM `Creates`;
20 • SELECT * FROM `Owns`;
21 • SELECT * FROM `Retrieves`;

```

Result Grid | Filter Rows:

	user_id	conversation_id
▶	1	1
	2	2
	3	3
	4	4
	5	5
	6	6
	7	7
	8	8
	9	9
	10	10
*	NULL	NULL

SCHEMAS

Filter objects

- bars-drinkers-beers
- ragdoll
- ragdoll_db**
 - Tables
 - access
 - admins
 - answers
 - audit_log
 - chunks
 - contains_query
 - contains_response
 - conversations
 - creates
 - documents
 - generates
 - has
 - manages
 - models
 - owns
 - payments
 - pays
 - prompts
 - queries
 - responses
 - retrieves
 - sets
 - splits_into
 - tiers
 - triggers
 - user_hashes
 - users
 - Views
 - Stored Procedures
 - Functions
- sys
- vu

10 • `SELECT * FROM `Conversations`;`

11 • `SELECT * FROM `Queries`;`

12 • `SELECT * FROM `Responses`;`

13 • `SELECT * FROM `Audit_Log`;`

14

15 • `SELECT * FROM `Triggers`;`

16 • `SELECT * FROM `Pays`;`

17 • `SELECT * FROM `Sets`;`

18 • `SELECT * FROM `Has`;`

19 • `SELECT * FROM `Creates`;`

20 • `SELECT * FROM `Owns`;`

21 • `SELECT * FROM `Access`;`

22 • `SELECT * FROM `Prompts`;`

23 • `SELECT * FROM `Generates`;`

24 • `SELECT * FROM `Retrieves`;`

25 • `SELECT * FROM `Answers`;`

26 • `SELECT * FROM `Contains_Response`;`

27 • `SELECT * FROM `Contains_Query`;`

28 • `SELECT * FROM `Manages`;`

29 • `SELECT * FROM `Splits_Into`;`

Result Grid

	tier_id	model_id
▶	1	1
	2	1
	3	1
	5	1
	3	2
	5	2
	4	3
	2	6
	6	7
	7	8
*	NULL	NULL

Navigator: **SQL File 11*** x database

SCHEMAS

Filter objects

- bars-drinkers-beers
- ragdoll
- ragdoll_db**
 - Tables
 - access
 - admins
 - answers
 - audit_log
 - chunks
 - contains_query
 - contains_response
 - conversations
 - creates
 - documents
 - generates
 - has
 - manages
 - models
 - owns
 - payments
 - pays
 - prompts
 - queries
 - responses
 - retrieves
 - sets
 - splits_into
 - tiers
 - triggers
 - user_hashes
 - users
 - Views
 - Stored Procedures
 - Functions
- sys
- vu

10 • `SELECT * FROM `Conversations`;`

11 • `SELECT * FROM `Queries`;`

12 • `SELECT * FROM `Responses`;`

13 • `SELECT * FROM `Audit_Log`;`

14

15 • `SELECT * FROM `Triggers`;`

16 • `SELECT * FROM `Pays`;`

17 • `SELECT * FROM `Sets`;`

18 • `SELECT * FROM `Has`;`

19 • `SELECT * FROM `Creates`;`

20 • `SELECT * FROM `Owns`;`

21 • `SELECT * FROM `Access`;`

22 • `SELECT * FROM `Prompts`;`

23 • `SELECT * FROM `Generates`;`

24 • `SELECT * FROM `Retrieves`;`

25 • `SELECT * FROM `Answers`;`

26 • `SELECT * FROM `Contains_Response`;`

27 • `SELECT * FROM `Contains_Query`;`

28 • `SELECT * FROM `Manages`;`

29 • `SELECT * FROM `Splits_Into`;`

Result Grid | Filter Rows: | Edit:

	query_id	model_id	prompted_at
▶	1	1	2026-07-04 14:01:05
	2	1	2026-07-04 14:31:05
	3	2	2026-07-04 15:01:05
	4	6	2026-07-04 15:31:05
	5	3	2026-07-04 16:01:05
	6	4	2026-07-04 16:31:05
	7	1	2026-07-04 17:01:05
	8	2	2026-07-04 17:31:05
	9	8	2026-07-04 18:01:05
	10	9	2026-07-04 18:31:05
*	NULL	NULL	NULL

Navigator

SCHEMAS

Filter objects

- bars-drinkers-beers
- ragdoll
- ragdoll_db**
 - Tables
 - access
 - admins
 - answers
 - audit_log
 - chunks
 - contains_query
 - contains_response
 - conversations
 - creates
 - documents
 - generates
 - has
 - manages
 - models
 - owns
 - payments
 - pays
 - prompts
 - queries
 - responses
 - retrieves
 - sets
 - splits_into
 - tiers
 - triggers
 - user_hashes
 - users
 - Views
 - Stored Procedures
 - Functions
- sys
- vu

SQL File 1 x database

10 • SELECT * FROM `Conversations`;

11 • SELECT * FROM `Queries`;

12 • SELECT * FROM `Responses`;

13 • SELECT * FROM `Audit\_Log`;

14

15 • SELECT * FROM `Triggers`;

16 • SELECT * FROM `Pays`;

17 • SELECT * FROM `Sets`;

18 • SELECT * FROM `Has`;

19 • SELECT * FROM `Creates`;

20 • SELECT * FROM `Owns`;

21 • SELECT * FROM `Access`;

22 • SELECT * FROM `Prompts`;

23 • SELECT * FROM `Generates`;

24 • SELECT * FROM `Retrieves`;

25 • SELECT * FROM `Answers`;

26 • SELECT * FROM `Contains\_Response`;

27 • SELECT * FROM `Contains\_Query`;

28 • SELECT * FROM `Manages`;

29 • SELECT * FROM `Splits\_Into`;

Result Grid

Filter Rows: Edit:

	model_id	response_id
▶	1	1
	1	2
	1	7
	2	3
	2	8
	3	5
	4	6
	6	4
	8	9
	9	10
*	NULL	NULL

Navigator

SCHEMAS

Filter objects

- bars-drinkers-beers
- ragdoll
- ragdoll_db**
 - Tables
 - access
 - admins
 - answers
 - audit_log
 - chunks
 - contains_query
 - contains_response
 - conversations
 - creates
 - documents
 - generates
 - has
 - manages
 - models
 - owns
 - payments
 - pays
 - prompts
 - queries
 - responses
 - retrieves
 - sets
 - splits_into
 - tiers
 - triggers
 - user_hashes
 - users
 - Views
 - Stored Procedures
 - Functions
- sys
- vu

SQL File 11* x database

10 • SELECT * FROM `Conversations`;

11 • SELECT * FROM `Queries`;

12 • SELECT * FROM `Responses`;

13 • SELECT * FROM `Audit\_Log`;

14

15 • SELECT * FROM `Triggers`;

16 • SELECT * FROM `Pays`;

17 • SELECT * FROM `Sets`;

18 • SELECT * FROM `Has`;

19 • SELECT * FROM `Creates`;

20 • SELECT * FROM `Owns`;

21 • SELECT * FROM `Access`;

22 • SELECT * FROM `Prompts`;

23 • SELECT * FROM `Generates`;

24 • SELECT * FROM `Retrieves`;

25 • SELECT * FROM `Answers`;

26 • SELECT * FROM `Contains\_Response`;

27 • SELECT * FROM `Contains\_Query`;

28 • SELECT * FROM `Manages`;

29 • SELECT * FROM `Splits\_Into`;

Result Grid | Filter Rows: | Edit:

	query_id	document_id	chunk_id
▶	1	1	1
	2	2	2
	3	3	3
	4	4	4
	5	5	5
	6	6	6
	7	7	7
	8	8	8
	9	9	9
	10	10	10
*	NULL	NULL	NULL

Navigator

SCHEMAS

Filter objects

- bars-drinkers-beers
- ragdoll
- ragdoll_db**
 - Tables
 - access
 - admins
 - answers
 - audit_log
 - chunks
 - contains_query
 - contains_response
 - conversations
 - creates
 - documents
 - generates
 - has
 - manages
 - models
 - owns
 - payments
 - pays
 - prompts
 - queries
 - responses
 - retrieves
 - sets
 - splits_into
 - tiers
 - triggers
 - user_hashes
 - users
 - Views
 - Stored Procedures
 - Functions
- sys
- vu

SQL File 11* x database

```

10 • SELECT * FROM `Conversations`;
11 • SELECT * FROM `Queries`;
12 • SELECT * FROM `Responses`;
13 • SELECT * FROM `Audit_Log`;
14
15 • SELECT * FROM `Triggers`;
16 • SELECT * FROM `Pays`;
17 • SELECT * FROM `Sets`;
18 • SELECT * FROM `Has`;
19 • SELECT * FROM `Creates`;
20 • SELECT * FROM `Owns`;
21 • SELECT * FROM `Access`;
22 • SELECT * FROM `Prompts`;
23 • SELECT * FROM `Generates`;
24 • SELECT * FROM `Retrieves`;
25 • SELECT * FROM `Answers`;
26 • SELECT * FROM `Contains_Response`;
27 • SELECT * FROM `Contains_Query`;
28 • SELECT * FROM `Manages`;
29 • SELECT * FROM `Splits_Into`;

```

Result Grid | Filter Rows: | Edit:

	query_id	response_id
▶	1	1
	2	2
	3	3
	4	4
	5	5
	6	6
	7	7
	8	8
	9	9
	10	10
*	NULL	NULL

SCHEMAS

Filter objects

- bars-drinkers-beers
- ragdoll
- ragdoll_db**
 - Tables
 - access
 - admins
 - answers
 - audit_log
 - chunks
 - contains_query
 - contains_response
 - conversations
 - creates
 - documents
 - generates
 - has
 - manages
 - models
 - owns
 - payments
 - pays
 - prompts
 - queries
 - responses
 - retrieves
 - sets
 - splits_into
 - tiers
 - triggers
 - user_hashes
 - users
 - Views
 - Stored Procedures
 - Functions
- sys
- vu

10 • SELECT * FROM `Conversations`;

11 • SELECT * FROM `Queries`;

12 • SELECT * FROM `Responses`;

13 • SELECT * FROM `Audit\_Log`;

14

15 • SELECT * FROM `Triggers`;

16 • SELECT * FROM `Pays`;

17 • SELECT * FROM `Sets`;

18 • SELECT * FROM `Has`;

19 • SELECT * FROM `Creates`;

20 • SELECT * FROM `Owns`;

21 • SELECT * FROM `Access`;

22 • SELECT * FROM `Prompts`;

23 • SELECT * FROM `Generates`;

24 • SELECT * FROM `Retrieves`;

25 • SELECT * FROM `Answers`;

26 • SELECT * FROM `Contains\_Response`;

27 • SELECT * FROM `Contains\_Query`;

28 • SELECT * FROM `Manages`;

29 • SELECT * FROM `Splits\_Into`;

Result Grid | Filter Rows: | Edit

	conversation_id	response_id
▶	1	1
	2	2
	3	3
	4	4
	5	5
	6	6
	7	7
	8	8
	9	9
	10	10
*	NULL	NULL

SCHEMAS

Filter objects

- bars-drinkers-beers
- ragdoll
- ragdoll_db**
 - Tables
 - access
 - admins
 - answers
 - audit_log
 - chunks
 - contains_query
 - contains_response
 - conversations
 - creates
 - documents
 - generates
 - has
 - manages
 - models
 - owns
 - payments
 - pays
 - prompts
 - queries
 - responses
 - retrieves
 - sets
 - splits_into
 - tiers
 - triggers
 - user_hashes
 - users
 - Views
 - Stored Procedures
 - Functions
- sys
- vu

10 • `SELECT * FROM `Conversations`;`

11 • `SELECT * FROM `Queries`;`

12 • `SELECT * FROM `Responses`;`

13 • `SELECT * FROM `Audit_Log`;`

14

15 • `SELECT * FROM `Triggers`;`

16 • `SELECT * FROM `Pays`;`

17 • `SELECT * FROM `Sets`;`

18 • `SELECT * FROM `Has`;`

19 • `SELECT * FROM `Creates`;`

20 • `SELECT * FROM `Owns`;`

21 • `SELECT * FROM `Access`;`

22 • `SELECT * FROM `Prompts`;`

23 • `SELECT * FROM `Generates`;`

24 • `SELECT * FROM `Retrieves`;`

25 • `SELECT * FROM `Answers`;`

26 • `SELECT * FROM `Contains_Response`;`

27 • `SELECT * FROM `Contains_Query`;`

28 • `SELECT * FROM `Manages`;`

29 • `SELECT * FROM `Splits_Into`;`

Result Grid | Filter Rows: | Edit:

	conversation_id	query_id
▶	1	1
	2	2
	3	3
	4	4
	5	5
	6	6
	7	7
	8	8
	9	9
	10	10
*	NULL	NULL

SCHEMAS

Filter objects

- bars-drinkers-beers
- ragdoll
- ragdoll_db**
 - Tables
 - access
 - admins
 - answers
 - audit_log
 - chunks
 - contains_query
 - contains_response
 - conversations
 - creates
 - documents
 - generates
 - has
 - manages
 - models
 - owns
 - payments
 - pays
 - prompts
 - queries
 - responses
 - retrieves
 - sets
 - splits_into
 - tiers
 - triggers
 - user_hashes
 - users
 - Views
 - Stored Procedures
 - Functions
- sys
- vu

SQL Editor

Limit to 5000

```

10 • SELECT * FROM `Conversations`;
11 • SELECT * FROM `Queries`;
12 • SELECT * FROM `Responses`;
13 • SELECT * FROM `Audit_Log`;
14
15 • SELECT * FROM `Triggers`;
16 • SELECT * FROM `Pays`;
17 • SELECT * FROM `Sets`;
18 • SELECT * FROM `Has`;
19 • SELECT * FROM `Creates`;
20 • SELECT * FROM `Owns`;
21 • SELECT * FROM `Access`;
22 • SELECT * FROM `Prompts`;
23 • SELECT * FROM `Generates`;
24 • SELECT * FROM `Retrieves`;
25 • SELECT * FROM `Answers`;
26 • SELECT * FROM `Contains_Response`;
27 • SELECT * FROM `Contains_Query`;
28 • SELECT * FROM `Manages`;
29 • SELECT * FROM `Splits_Into`;

```

Result Grid | Filter Rows: | Edit:

	admin_user_id	document_id	managed_at
▶	11	1	2026-07-03 09:00:00
	12	2	2026-07-03 09:10:00
	13	3	2026-07-03 09:20:00
	14	4	2026-07-03 09:30:00
	15	5	2026-07-03 09:40:00
	16	6	2026-07-03 10:00:00
	17	7	2026-07-03 10:10:00
	18	8	2026-07-03 10:20:00
	19	9	2026-07-03 10:30:00
	20	10	2026-07-03 10:40:00
*	NULL	NULL	NULL

SCHEMAS

Filter objects

- bars-drinkers-beers
- ragdoll
- ragdoll_db**
 - Tables
 - access
 - admins
 - answers
 - audit_log
 - chunks
 - contains_query
 - contains_response
 - conversations
 - creates
 - documents
 - generates
 - has
 - manages
 - models
 - owns
 - payments
 - pays
 - prompts
 - queries
 - responses
 - retrieves
 - sets
 - splits_into
 - tiers
 - triggers
 - user_hashes
 - users
 - Views
 - Stored Procedures
 - Functions
- sys
- vu

10 • SELECT * FROM `Conversations`;

11 • SELECT * FROM `Queries`;

12 • SELECT * FROM `Responses`;

13 • SELECT * FROM `Audit\_Log`;

14

15 • SELECT * FROM `Triggers`;

16 • SELECT * FROM `Pays`;

17 • SELECT * FROM `Sets`;

18 • SELECT * FROM `Has`;

19 • SELECT * FROM `Creates`;

20 • SELECT * FROM `Owns`;

21 • SELECT * FROM `Access`;

22 • SELECT * FROM `Prompts`;

23 • SELECT * FROM `Generates`;

24 • SELECT * FROM `Retrieves`;

25 • SELECT * FROM `Answers`;

26 • SELECT * FROM `Contains\_Response`;

27 • SELECT * FROM `Contains\_Query`;

28 • SELECT * FROM `Manages`;

29 • SELECT * FROM `Splits\_Into`;

Result Grid | Filter Rows: | Edit: |

	document_id	chunk_id
▶	1	1
	2	2
	3	3
	4	4
	5	5
	6	6
	7	7
	8	8
	9	9
	10	10
*	NULL	NULL